AI Talent Hub ИТМО

Практикум по исследовательской и разработческой деятельности в ML.

Занятие 4.

1. План занятия

1. Feature engineering vs representation learning

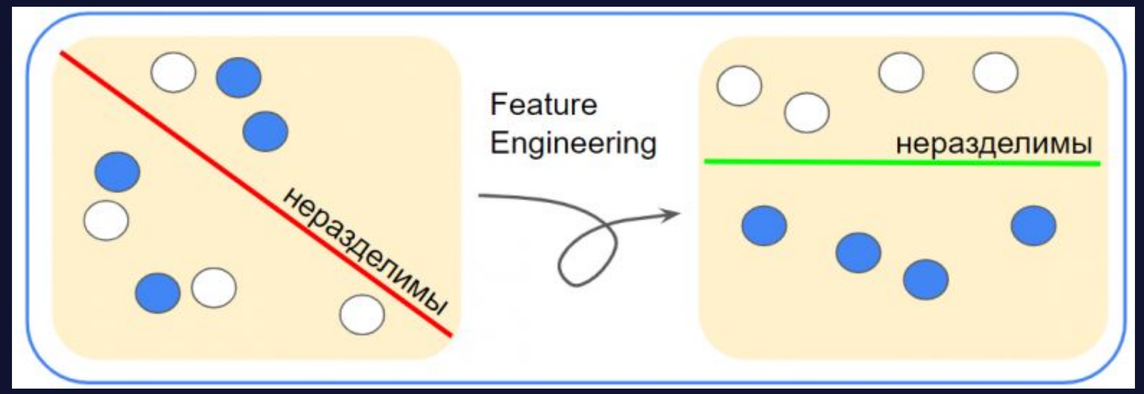
2. Регуляризация/аугментации/семплинг/балансировка

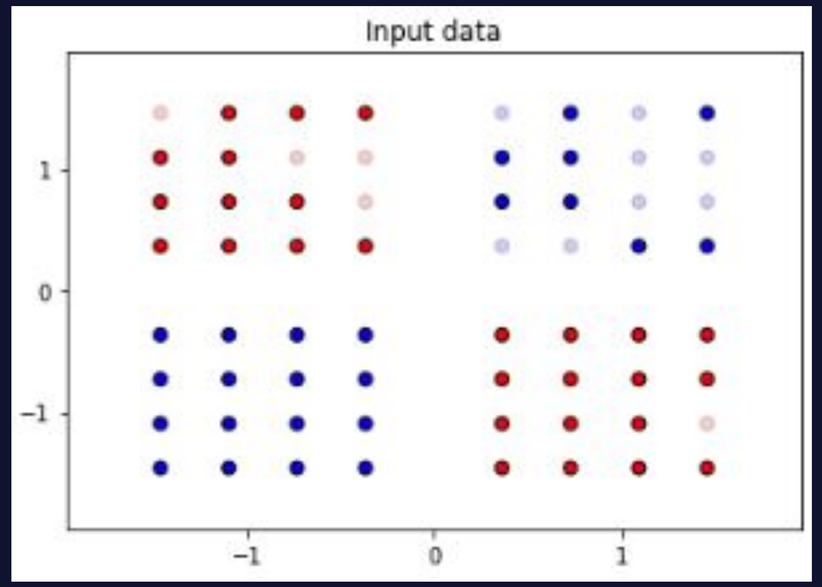
3. Hyperparameter optimization

https://habr.com/ru/companies/ruvds/articles/680498/

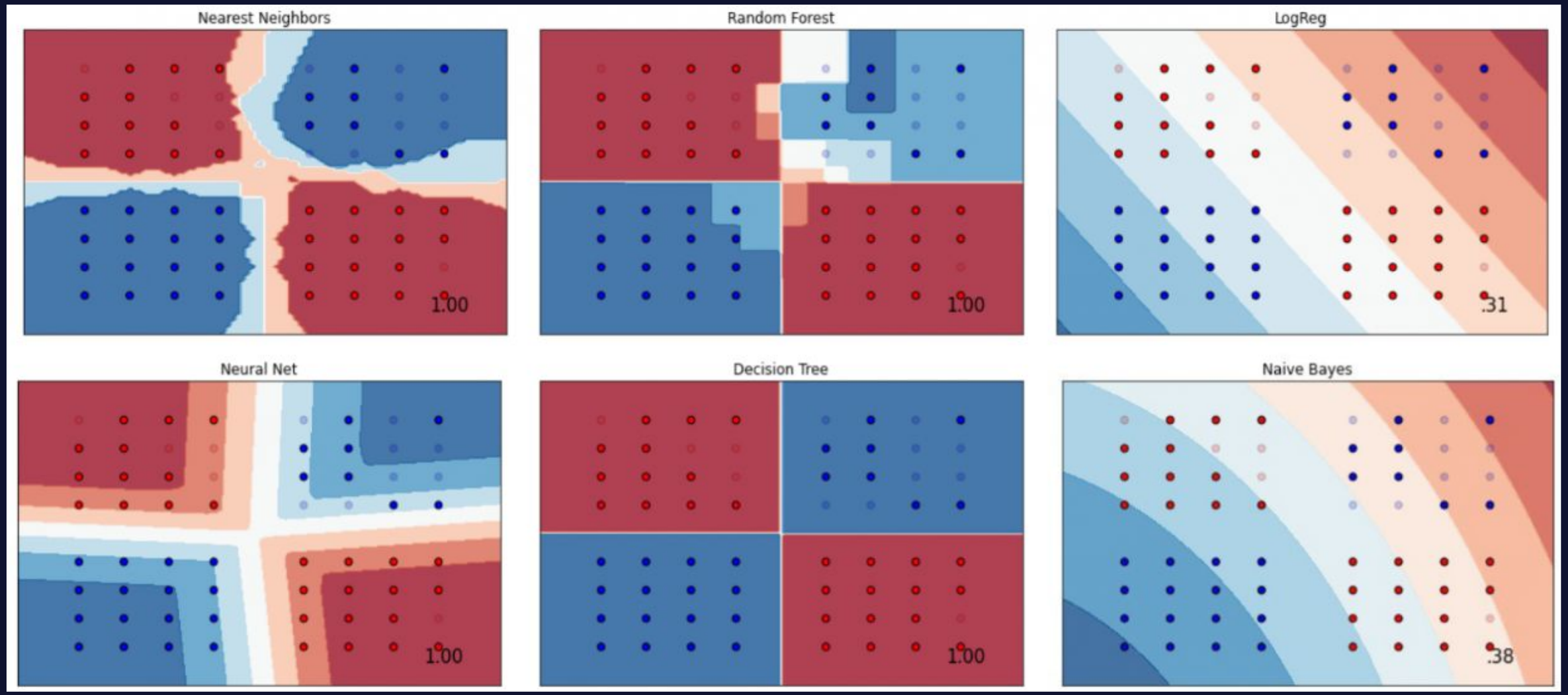
2. Features (признаки)

- ***Признаки (фичи, или features)*** — *отдельные характеристики или свойства данных, которые используются моделью для обучения и предсказания.*
- ***Пример***:
  В задаче предсказания цены дома площадь, количество комнат и возраст здания будут признаками, которые модель использует для определения стоимости.

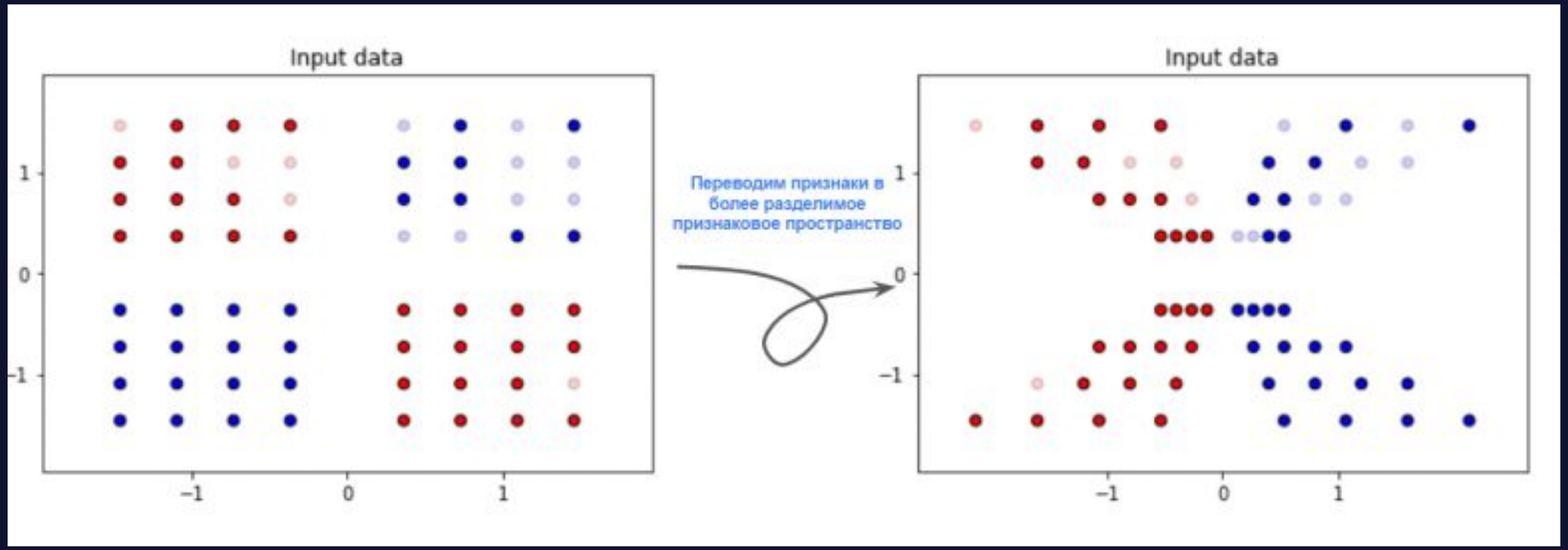




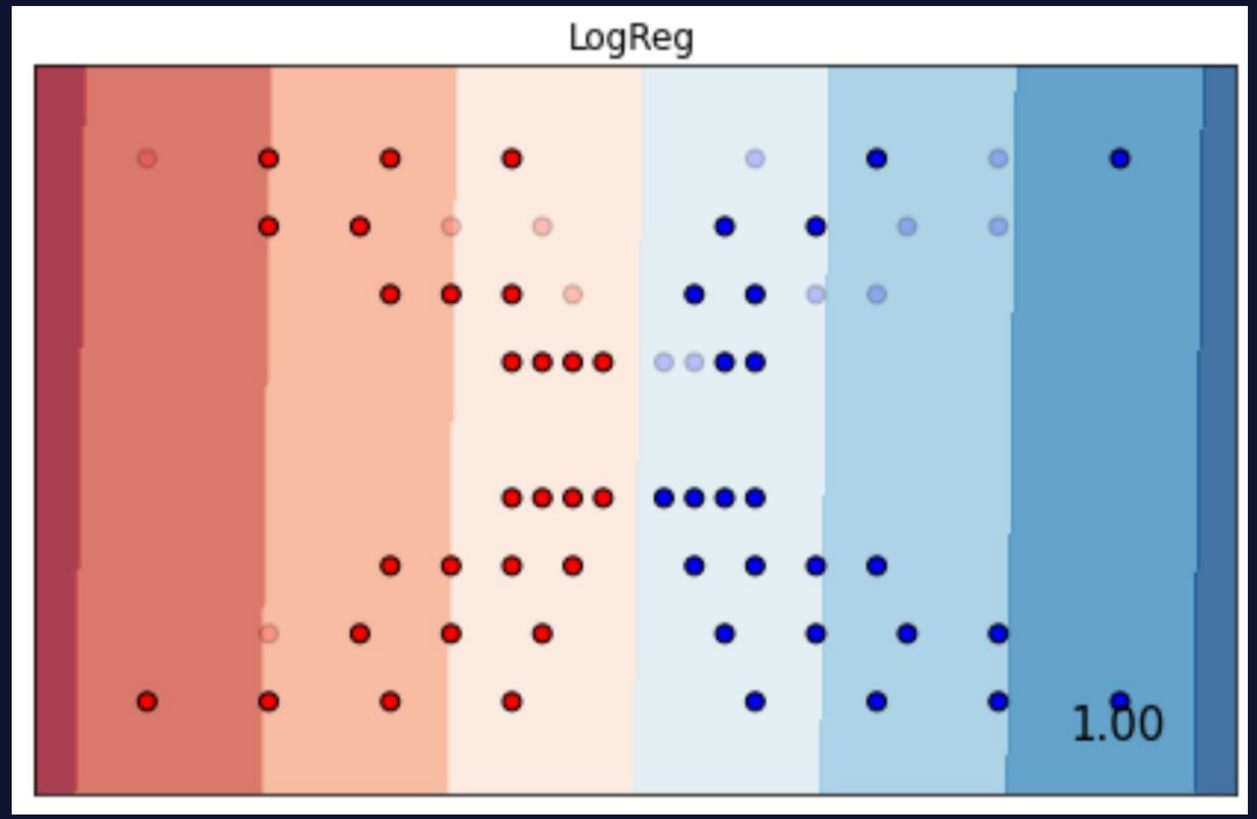
2. Features (признаки)



2. Features (признаки)

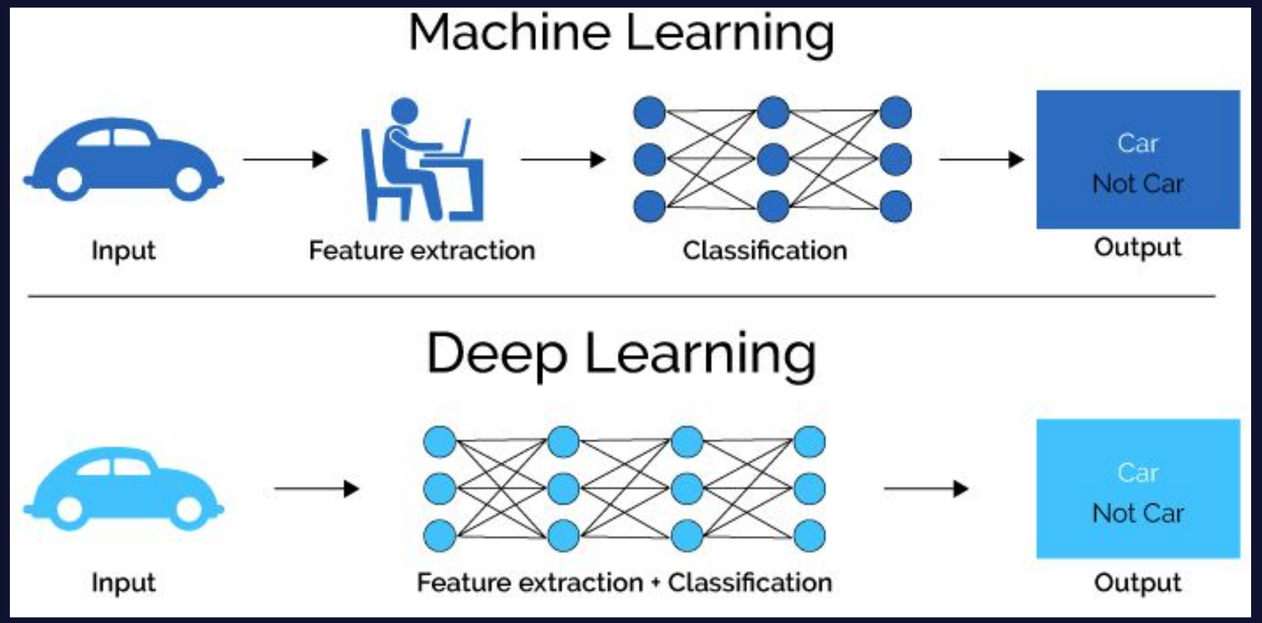


2. Features (признаки)



- *Мы заменили одну координату произведением координат.*
- *Такое преобразование признаков, пример элементарного FE позволил нам решить задачу с помощью простейшей модели логистической регресии*

2. Features (признаки)

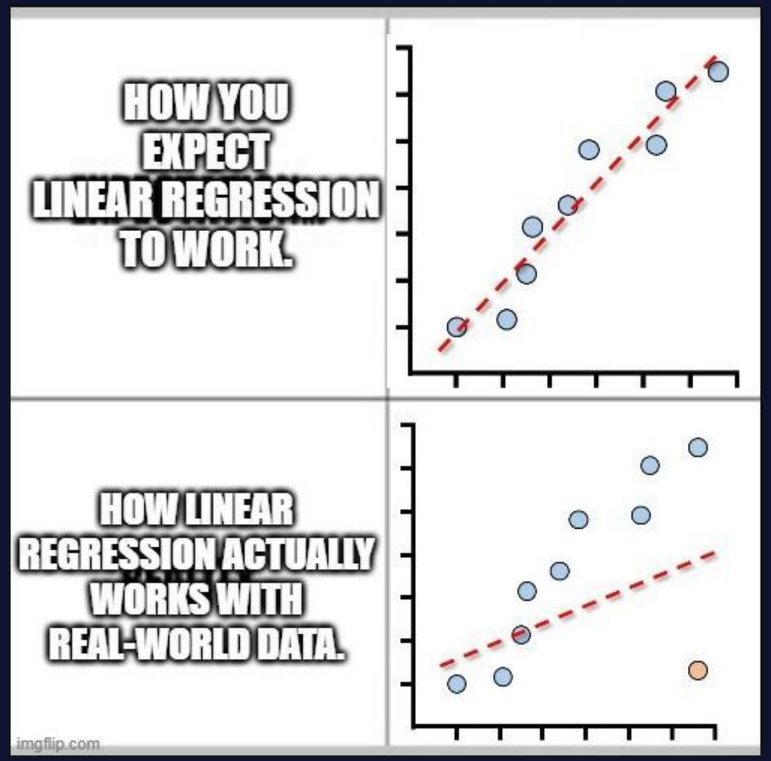


- *Картинка не всегда справедлива, скорее исторически сложившееся распределение подходов*
- *Такое преобразование признаков, пример элементарного FE позволил нам решить задачу с помощью простейшей модели логистической регресии*

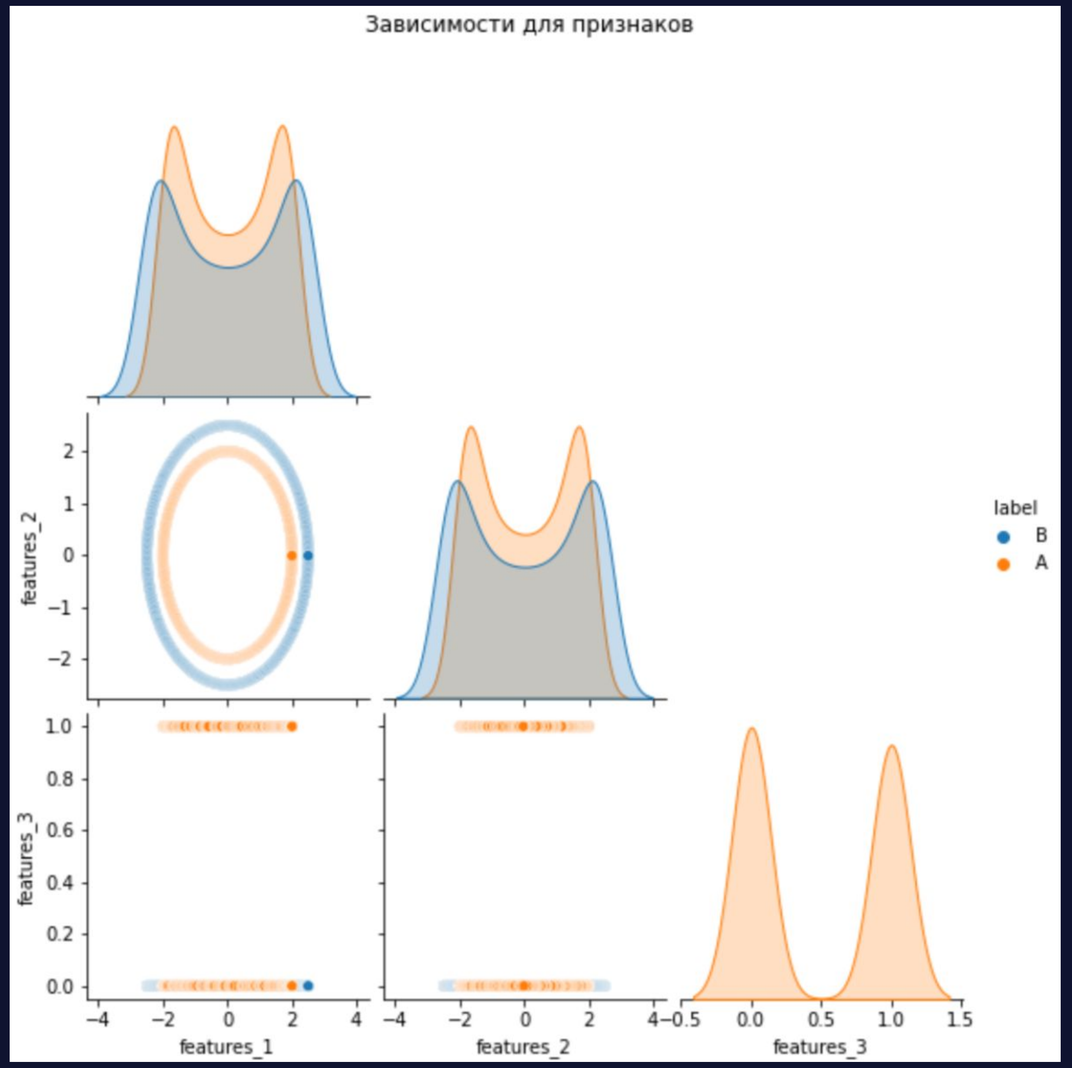
2. Features (признаки)

features _1	features _2	features _3	label	
0	2.5	0.00	58	B
1	2.5	0.02	61	A
2	2.5	0.05	33	B
3	2.5	0.07	89	B
4	2.5	0.10	58	A

- *Хотим получить точность 100% на линейной модели*
- *Точность при обучении “as is”:* ***0.489***

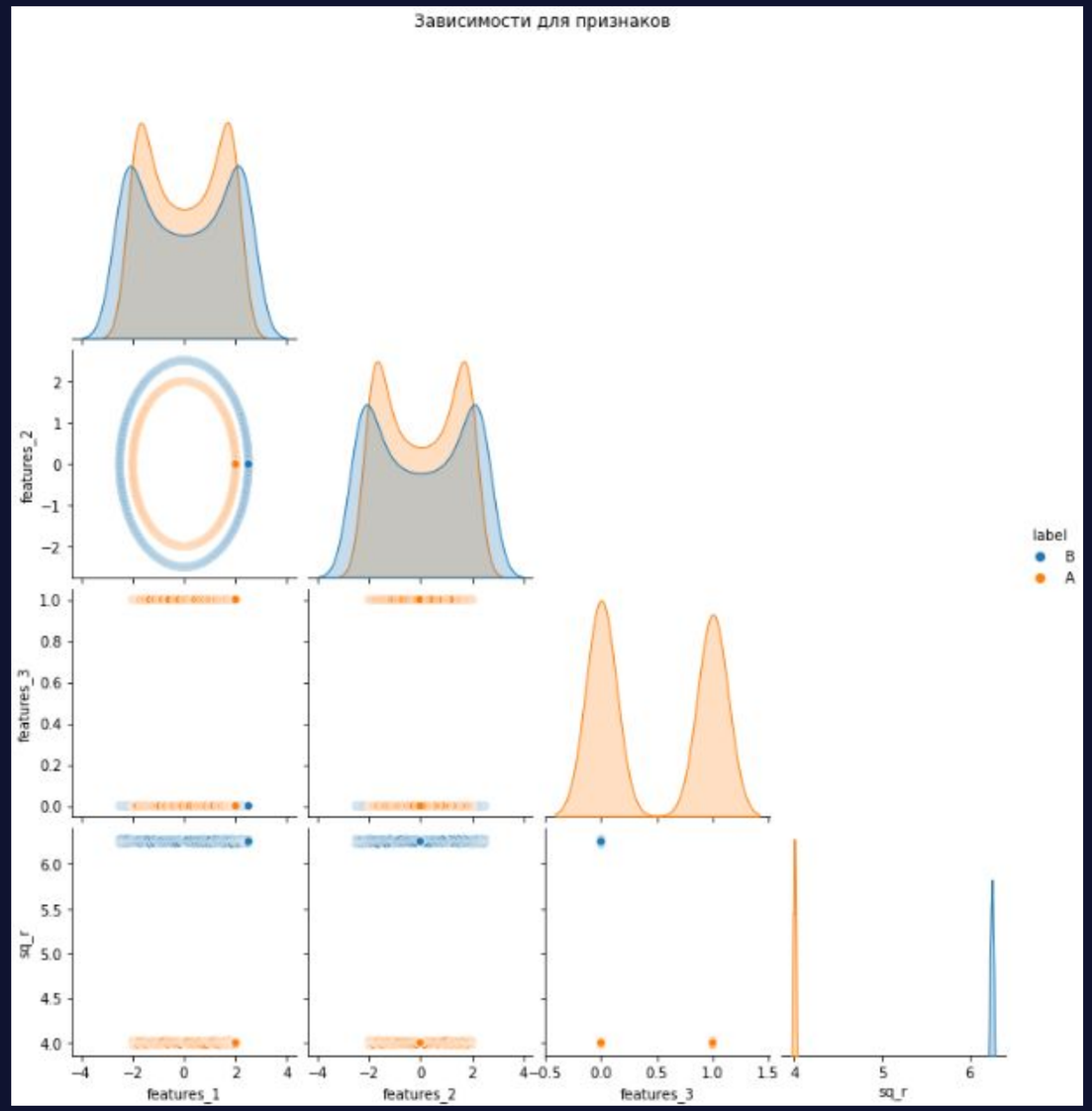


2. Features (признаки)




```
sns.pairplot( # матрица диаграмм рассеяния из Seaborn
data = df, # данные
hue = 'label', # имя столбца в данных для раскраски меток
corner = True # матрица симметрична, не будем выводить лишние графики
).fig.suptitle( # настройки заголовка
'Зависимости для признаков', # заголовок
y = 1.08 # расположение
);
```

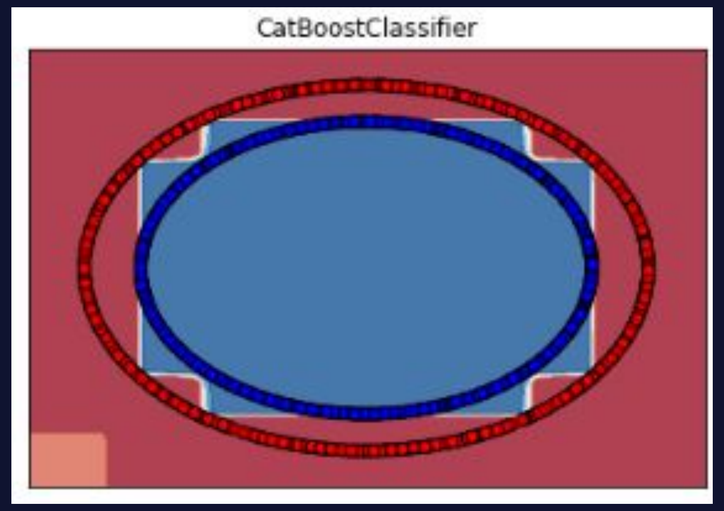
2. Features (признаки)




```
sns.pairplot( # матрица диаграмм рассеяния из Seaborn
data = df, # данные
hue = 'label', # имя столбца в данных для раскраски меток
corner = True # матрица симметрична, не будем выводить лишние графики
).fig.suptitle( # настройки заголовка
'Зависимости для признаков', # заголовок
y = 1.08 # расположение
);
```

$$(x-a)^2+(y-b)^2=r^2$$

```
df['sq_r'] = (df['features_1'] ** 2 + df['features_2'] ** 2)
```



3. Representation Learning

- ***Representation learning или Feature Learning - это*** *набор техник, которые позволяют системе автоматически обнаружить представления, необходимые для выявления признаков или классификации исходных (сырых) данных.*
- ***Пример****:*
  в задачах машинного обучения, таких как классификация, часто требуется вход, который удобно обрабатывать математически и вычислительно. Однако реальные данные, такие как изображения, видео и данные датчиков, не поддаются алгоритмическому определению специфичных признаков.

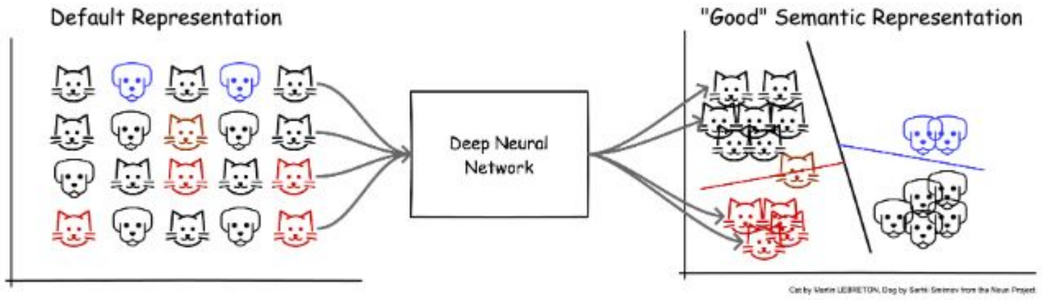
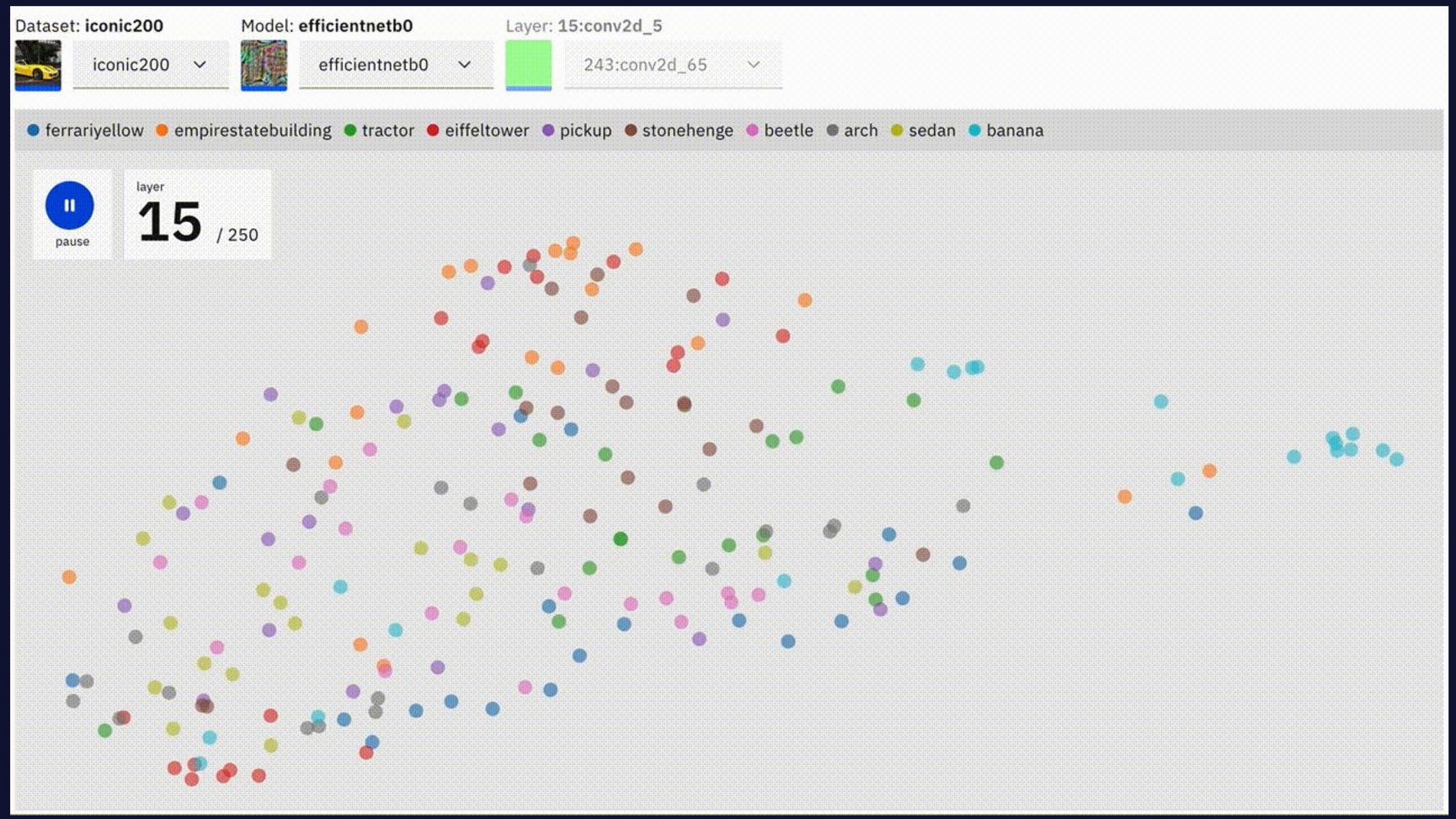


Figure 2: a.) Default representation of data (e.g., raw pixel values) do not encode semantic meaning b.) Features or representations provided by a good DNN model should encode meaning related to the current task (in this case, classification). With this representation, similar data items are *closer* to each other; the original non-linear problem is now linearly separable, and therefore easier to solve.

3. Representation Learning



$$\mathcal{L}(\theta, \phi) = \int_{i \in \mathcal{X}} D_{\mathrm{KL}}\left(p_\theta(\cdot|i) || q_\phi(\cdot|i)\right) = \int_{i \in \mathcal{X}} \int_{j \in \mathcal{X}} p_\theta(j|i) \log \frac{p_\theta(j|i)}{q_\phi(j|i)}.$$

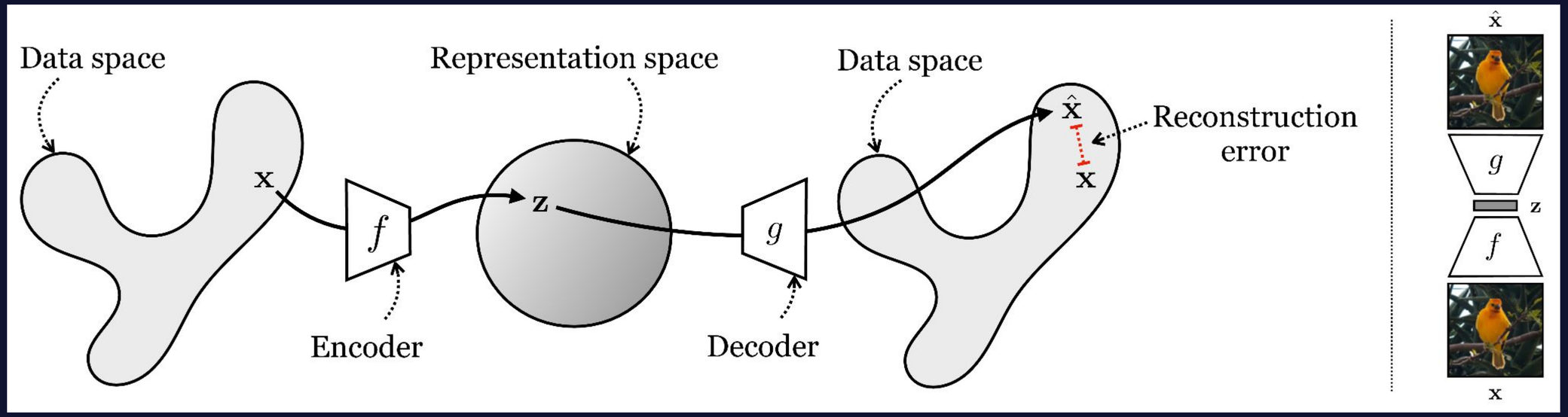
3. Representation Learning

Learned Representation \ Supervisory Signal	Gaussian	Student-T	Identity	Graph Kernel Weights	Uniform over K-Neighbors	Uniform over Positive Pairs	Cross-Modal Pairs	Uniform over Classes	Data-Label Pairs
Gaussian	SNE [Hinton 2002] X-Sample CL [Sobal 2025]	Dual t-SNE		SNE Graph Embeddings	SNE with Uniform Affinities LGSimCLR [El Banani 2023]	InfoNCE [Bachman 2019] SimCLR [Chen 2020] MoCoV3 [Chen 2021]	CLIP [Radford 2021] CMC [Tian 2020]	SupCon [Khosla 2020]	Cross Entropy [Good 1963]
Gaussian $\sigma \to \infty$			PCA [Pearson 1901]			VI-Reg [Bardes 2021]	Average Margin CLIP	Average Margin SupCon	
Gaussian $\sigma \to 0$						Triplet Loss [Schroff 2015]	Triplet CLIP	Triplet SupCon	Error rate
Student-T	t-SNE [Van der Maaten 2008]	Doubly t-SNE		t-SNE Graph Embedding	t-SNE with Uniform Affinities	t-SimCNE [Böhm 2023] t-SimCLR [Hu 2023]	t-CLIP	t-SupCon	Harmonic Loss [Baek 2025]
Uniform on Clusters	K-Means [Macqueen 1967]	t K-Means		Normalized Cuts [Shi 2000]	DCD [Yang 2012]	InfoNCE Clustering [Ours]		Supervised Clustering	

Dimensionality Reduction · Cluster Learning · Unimodal SSL · Multimodal SSL · Supervised Learning · Interpretation of Gaps

3. Representation Learning

- *К какой representation мы стремимся?*



- *В обучении признакам с учителем обучение происходит с помощью помеченных входных данных. Примерами являются нейронные сети с учителем, многослойный перцептрон и словарное обучение (с учителем).*
- *В обучении признакам без учителя обучение происходит с помощью непомеченных входных данных. Примерами являются словарное обучение, анализ независимых компонент, автокодировщики, разложение матриц[2] и различные формы кластеризации[3][4][5].*

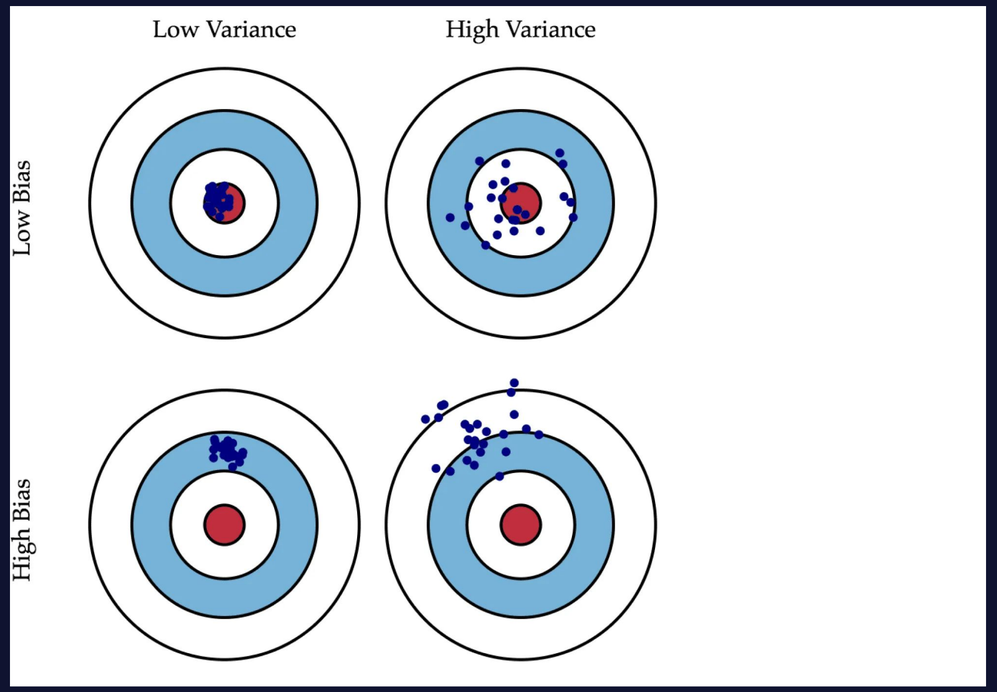
4. Регуляризация

- *Variance - Зависимость выхода модели от тренировочной выборки.*

$$\mathbb{V}\mathrm{ar}_X[a(x,X)] = \mathbb{E}_X\left[a(x,X) - \mathbb{E}_X[a(x,X)]\right]^2$$

- *Bias - показывает, насколько хорошо с помощью данного алгоритма можно приблизить истинную зависимость*

$$\mathrm{bias}_X a(x,X) = f(x) - \mathbb{E}_X[a(x,X)]$$



- *Регуляризация - набор методов, используемых в машинном обучении для предотвращения чрезмерной сложности моделей и улучшения их способности к обобщению новых, ранее не виданных данных. Методы для борьбы с Variance составляющей ошибки модели.*

4. Регуляризация

L1 регуляризация

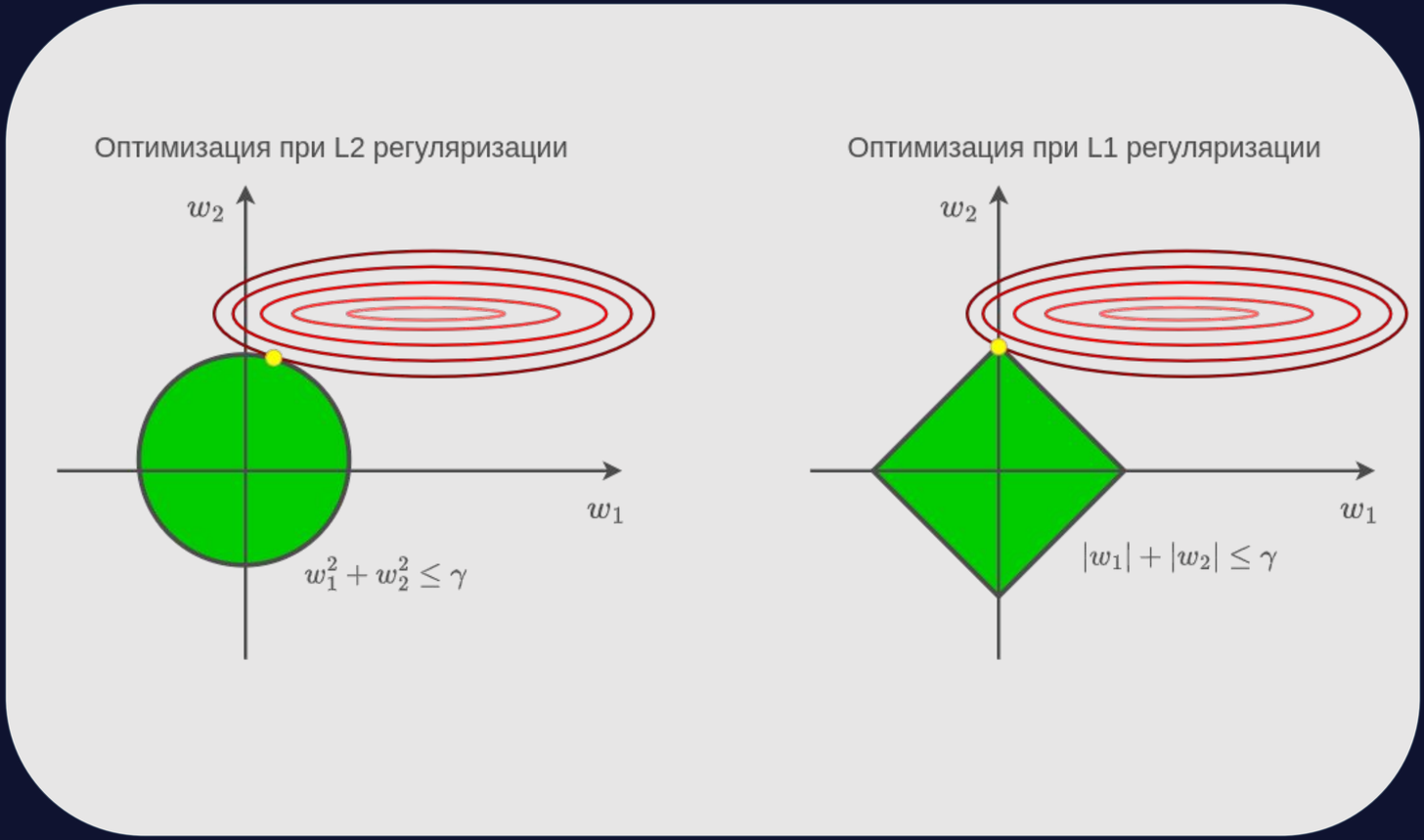
$$L_1 = \boxed{\sum_i (y_i - y(t_i))^2} + \boxed{\lambda \sum_i |a_i|}.$$

ошибка модели по метрике *штраф*

L2 регуляризация

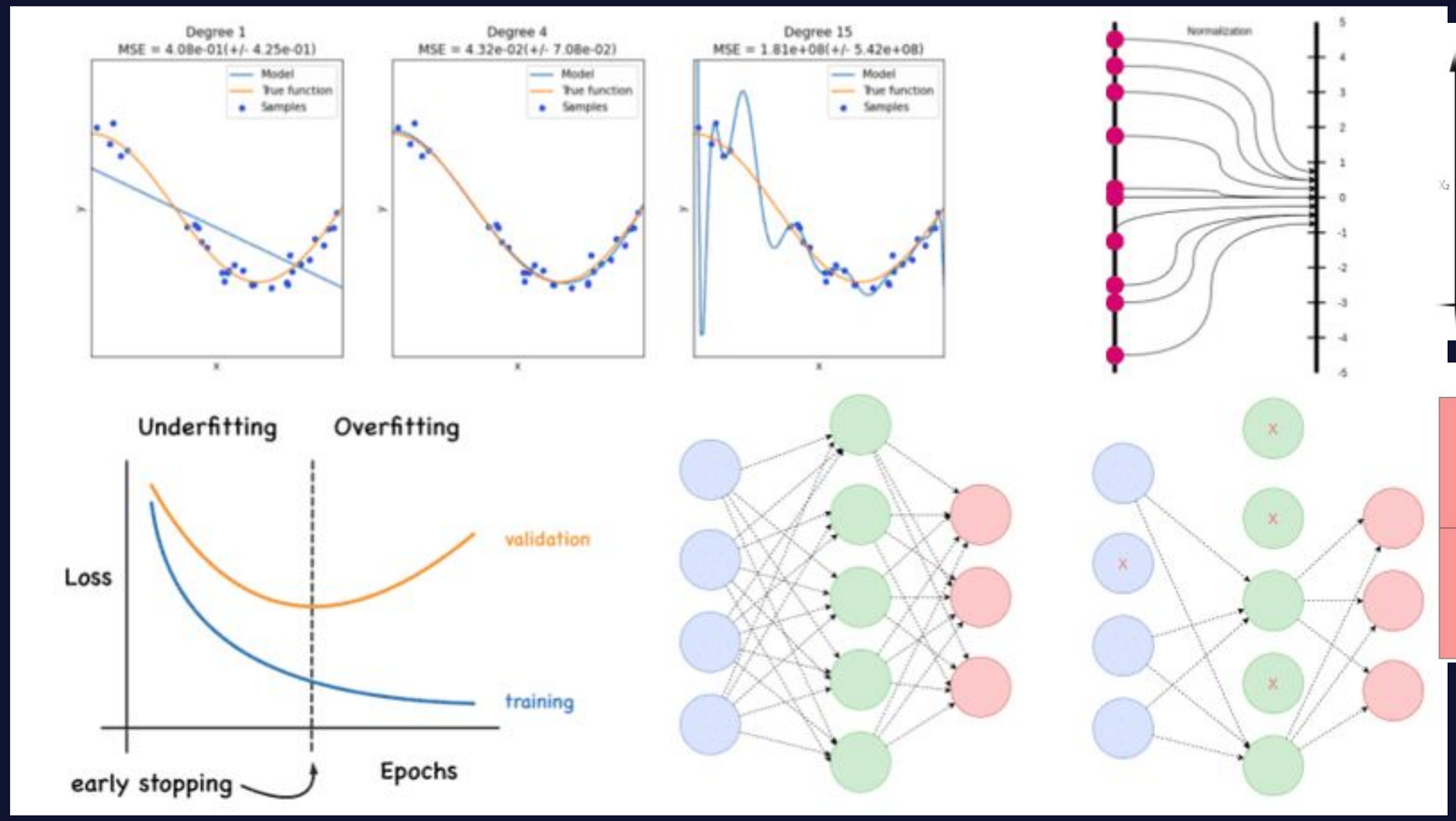
$$L_2 = \boxed{\sum_i (y_i - y(t_i))^2} + \boxed{\lambda \sum_i a_i^2}.$$

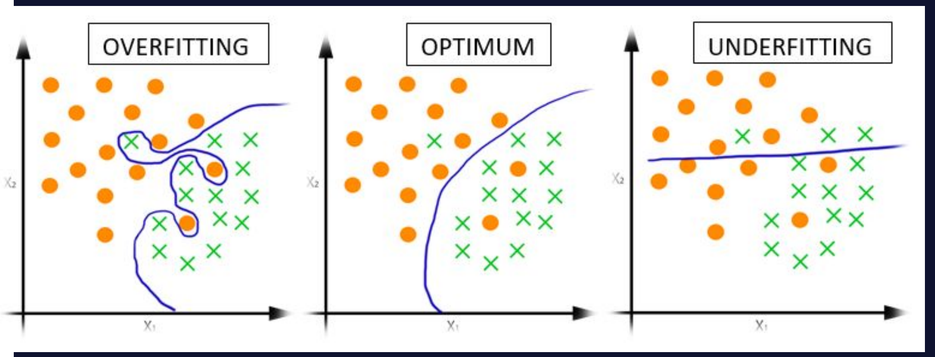
ошибка модели по метрике *штраф*

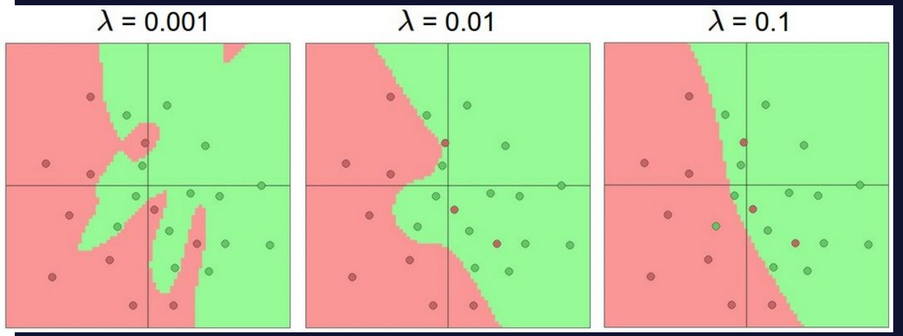


- *Регуляризация - набор методов, используемых в машинном обучении для предотвращения чрезмерной сложности моделей и улучшения их способности к обобщению новых, невиданных данных. Методы для борьбы с Variance составляющей ошибки модели.*

4. Регуляризация





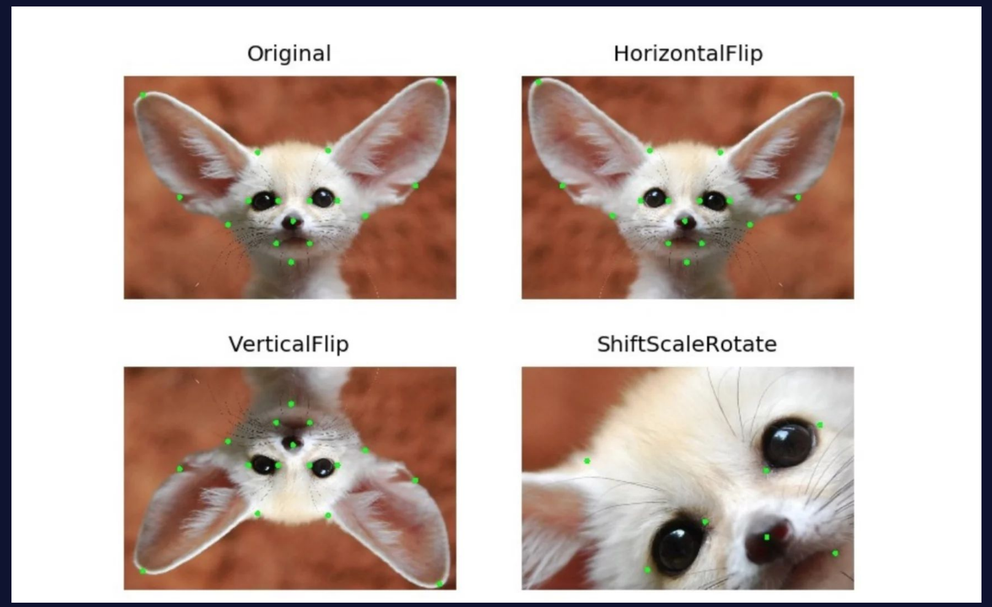


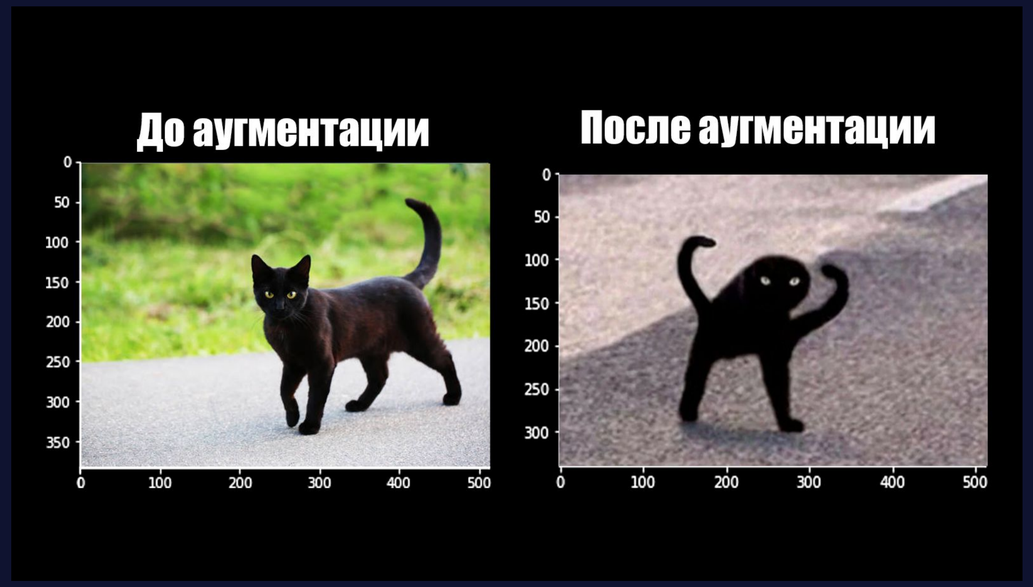
4. Регуляризация

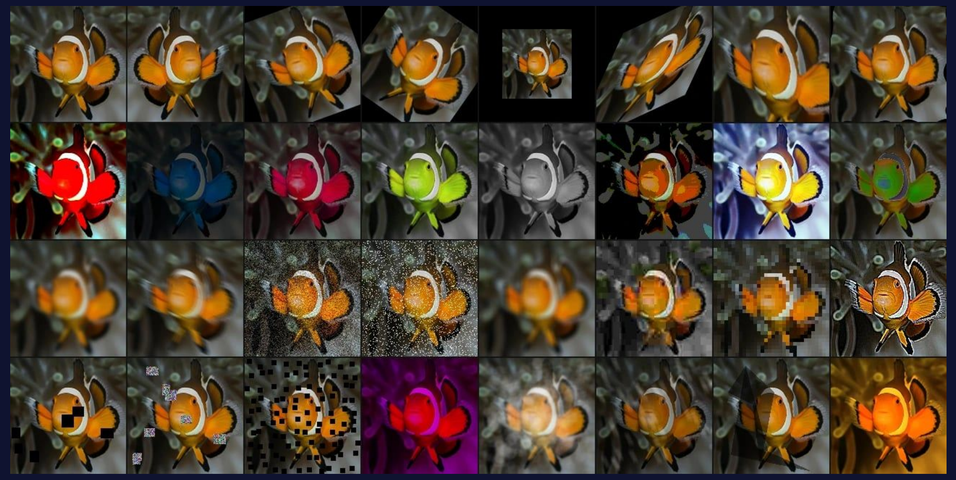
Метод	Идея	Где применяется	Плюсы	Минусы
L2 (Weight Decay)	штраф за большие веса (($\|w\|^2$))	почти везде (DL, linear, boosting)	стабилизирует обучение	не зануляет веса
L1	штраф за ($\|w\|$)	линейные модели	разреженность (feature selection)	нестабильность
Dropout	случайно отключает нейроны	нейросети	снижает co-adaptation	замедляет обучение
BatchNorm	нормализация активаций	нейросети	ускоряет и стабилизирует	не чистая регуляризация
Data Augmentation	увеличивает данные	CV, audio, NLP	уменьшает overfit	можно исказить данные
Early Stopping	остановка по валидации	любые модели	простой и эффективный	зависит от split
Label Smoothing	сглаживает таргеты	классификация	уменьшает overconfidence	может вредить точности
Noise Injection	добавление шума	DL, tabular	повышает устойчивость	трудно тюнить
Mixup / CutMix	смешивание примеров	CV, иногда NLP	сильная регуляризация	сложнее обучение
Прунинг	зануляет веса (не нейроны)	нейросети	сильнее dropout	редко используется

5. Аугментации

- ***Аугментации -*** *это процесс искусственного генерирования новых данных на основе существующих, который используется преимущественно для обучения новых моделей машинного обучения.*



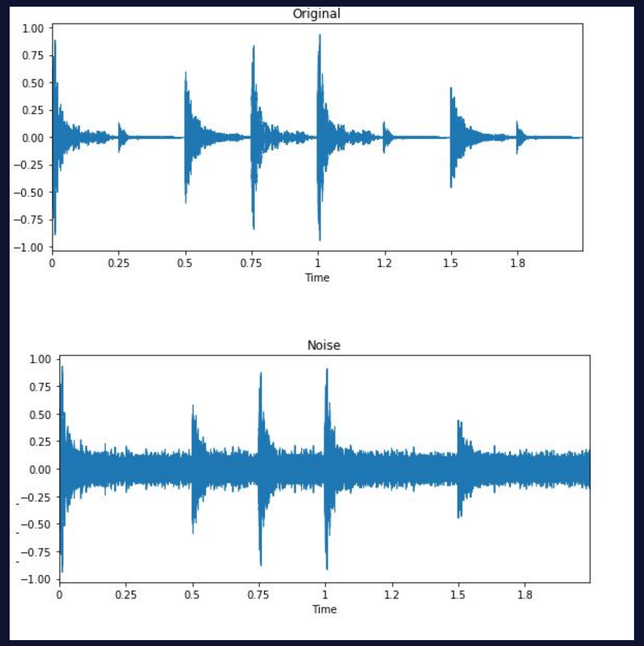


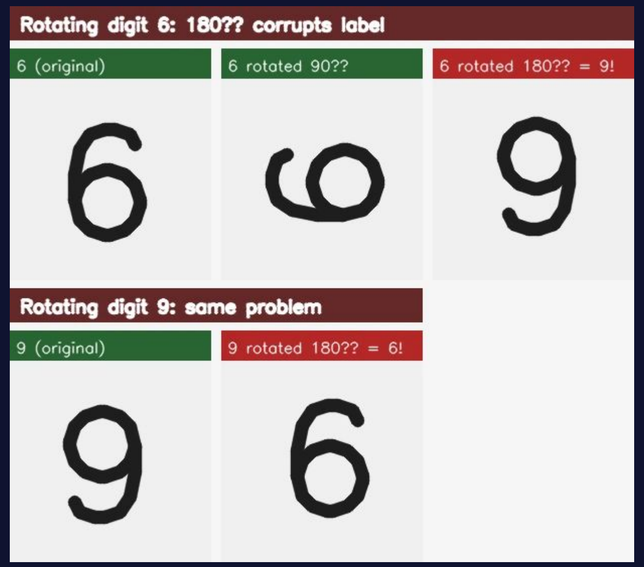


5. Аугментации

- ***Первый уровень -*** *Аугментация с “правдоподобными” и основанными на реальных зависимостях вариации*

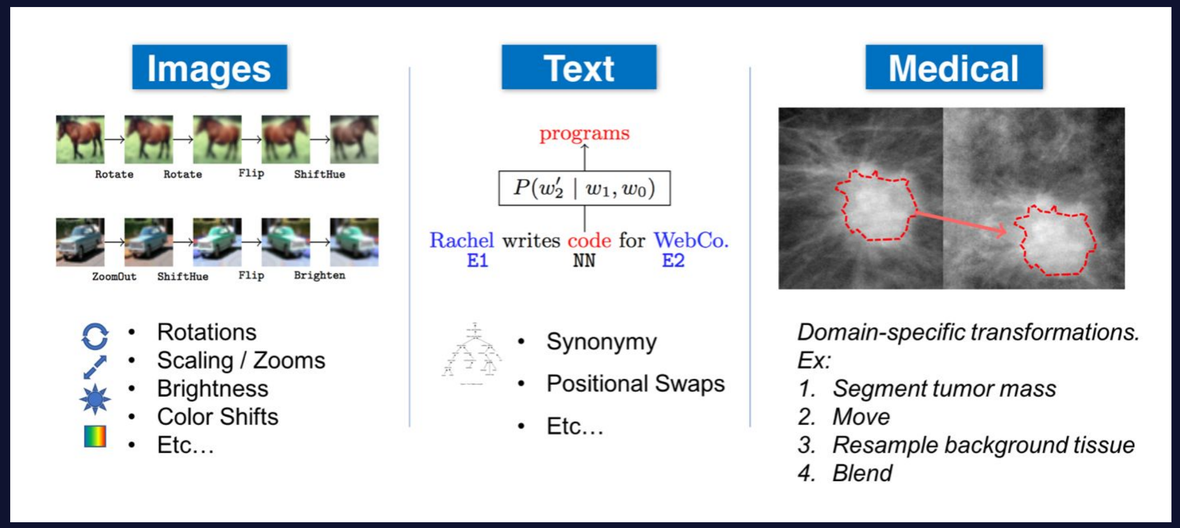
- ***Второй уровень*** *- намеренно усложняем задачу, чтобы модель учила более сильные признаки*

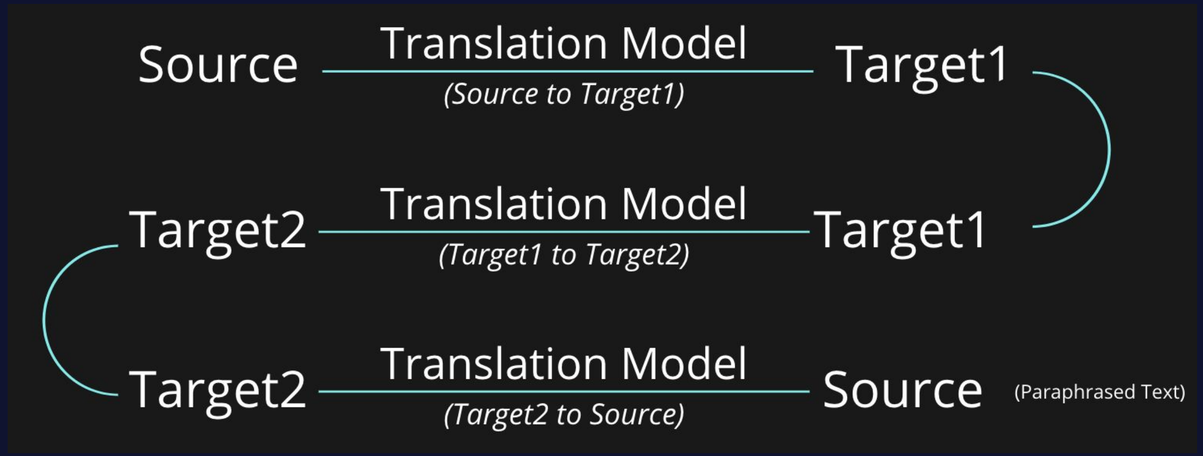




5. Аугментации

- ***Аугментации*** *и их оптимальный подбор очень сильно зависят от домена данных, а также задачи, для решения которых они подбираются.*





5. Аугментации

Домен	Аугментация	Идея	Когда полезно	Риск
CV	Flip / Rotate	геометрические трансформации	объекты не зависят от ориентации	ломает семантику (текст, лица)
	Crop / Resize	случайные области	устойчивость к положению	потеря важного объекта
	Color jitter	яркость/контраст	разные условия освещения	нереалистичные цвета
	CutMix	склейка изображений	сильная регуляризация	шумные лейблы
	Mixup	линейная смесь	сглаживание decision boundary	сложнее обучение
	Gaussian noise	добавление шума	устойчивость к шуму	ухудшение качества
	Blur	размытие	устойчивость к качеству камеры	теряются детали

5. Аугментации (NLP)

Аугментация	Идея	Когда полезно	Риск
Synonym replacement	замена слов	вариативность языка	искажение смысла
Back translation	перевод туда-обратно	перефразирование	дорого
Random deletion	удаление слов	robustness	потеря смысла
Token masking	маскирование	pretraining-style	не всегда подходит
Noise (typos)	опечатки	пользовательский ввод	переусложнение

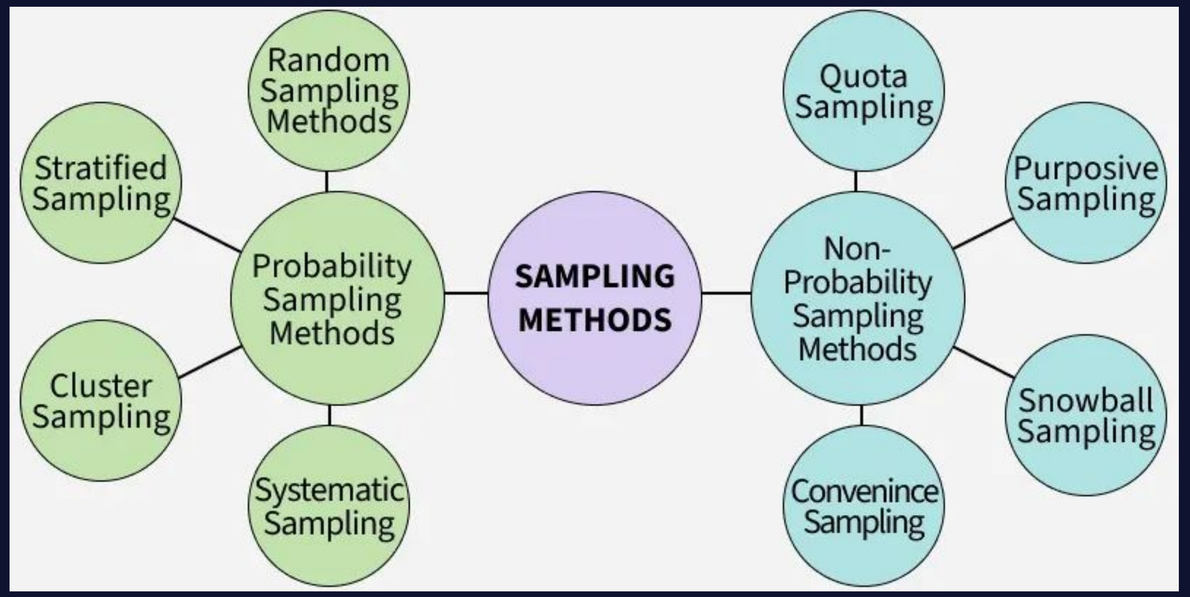
5. Аугментации (Audio)

Аугментация	Идея	Когда полезно	Риск
Time shift	сдвиг сигнала	инвариантность	почти безопасно
Noise injection	шум	реальные условия	сильный шум портит
Pitch shift	изменение высоты	разные голоса	искажение
Speed perturbation	скорость	разные темпы речи	искажение
SpecAugment	маскирование спектра	SOTA для ASR	требует тюнинга

5. Аугментации (Tabular)

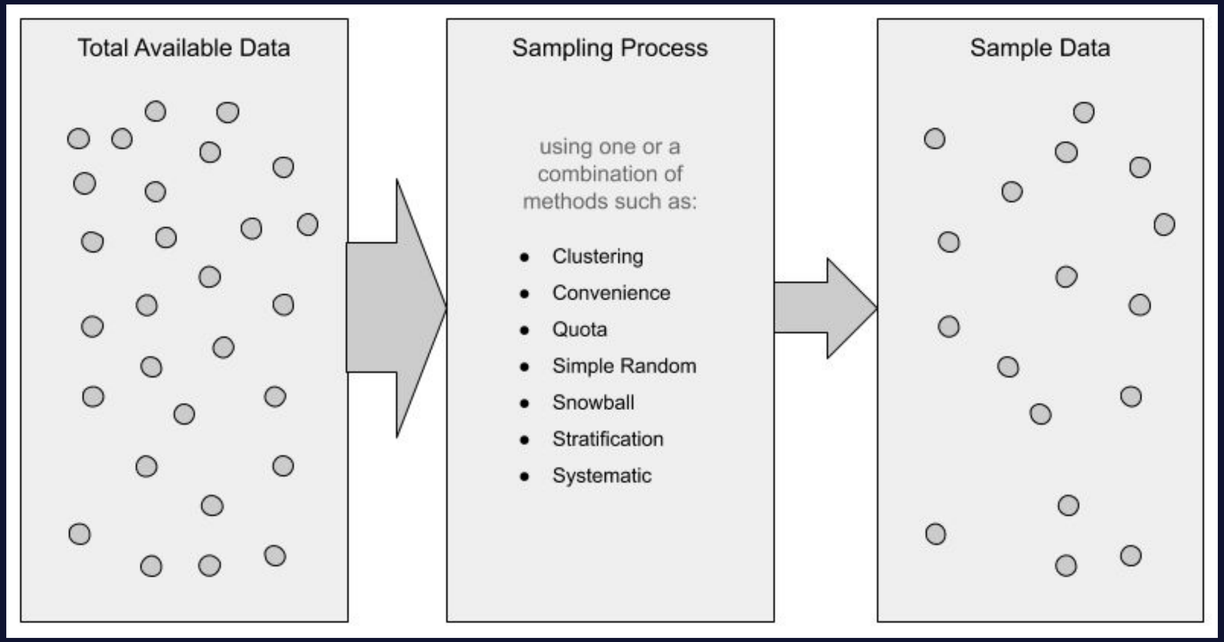
Аугментация	Идея	Когда полезно	Риск
Noise	шум в фичах	устойчивость	ломает распределение
SMOTE	синтетика minority	дисбаланс	переобучение
Feature dropout	удаление фич	robustness	потеря инфы
Permutation	перемешивание	анализ важности	не всегда корректно

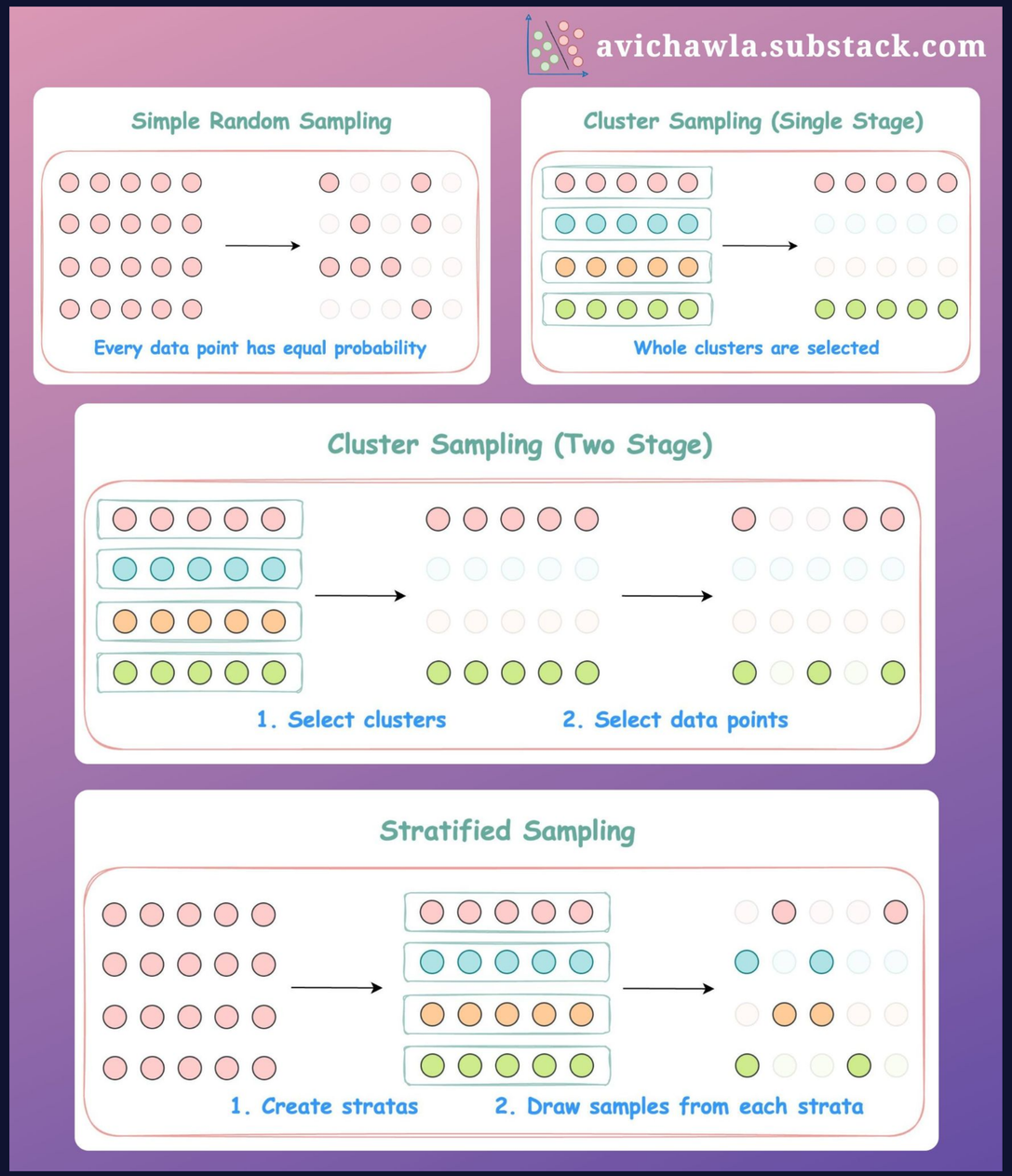
6. Семплинг



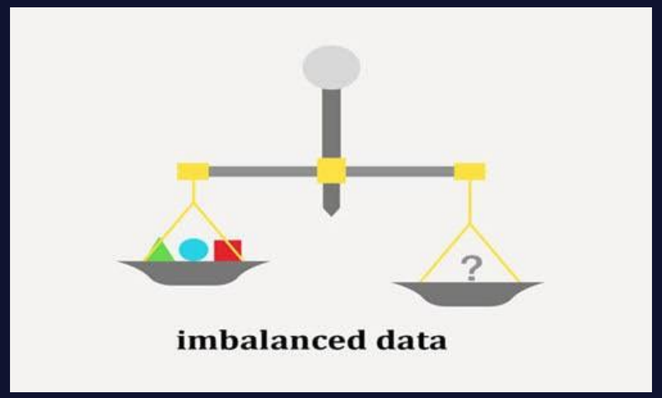
Что это	Зачем нужен	Где используется	Примеры
Выбор подмножества данных или изменение распределения выборки	Управление обучением и оценкой модели	обучение, валидация, работа с дисбалансом	train/test split, oversampling, undersampling

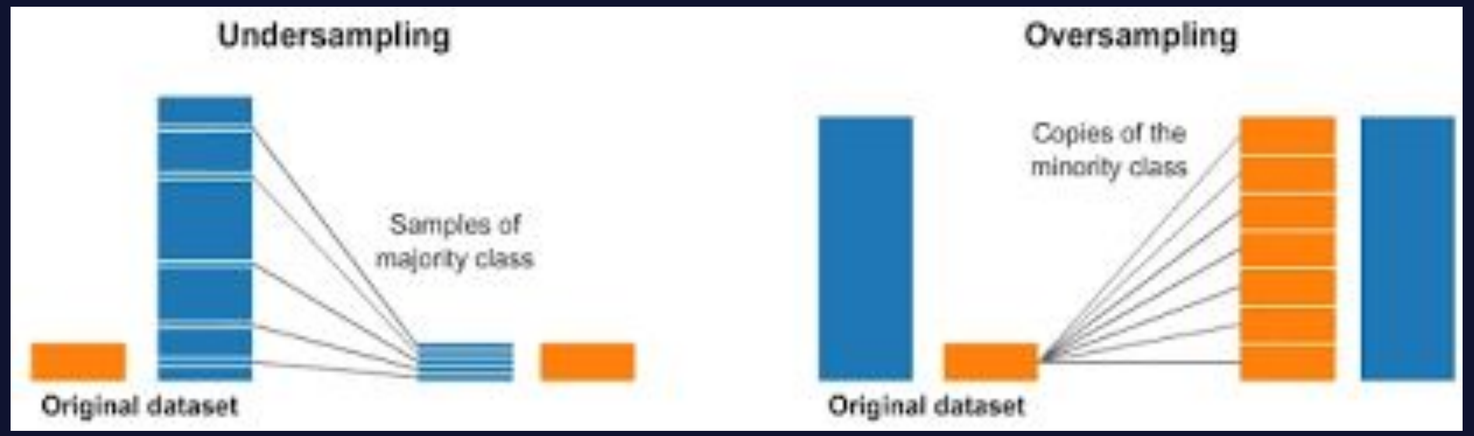
6. Семплинг и балансировка

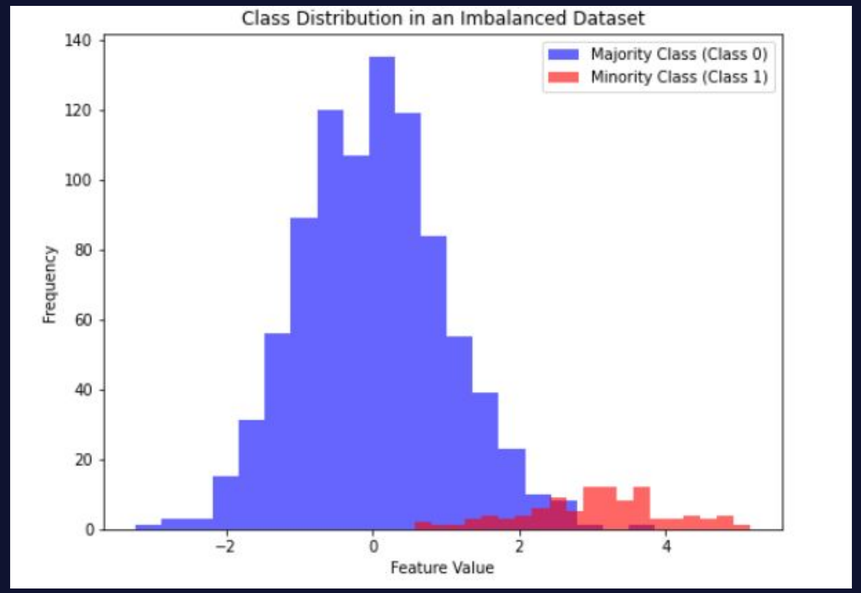




6. Семплинг и балансировка



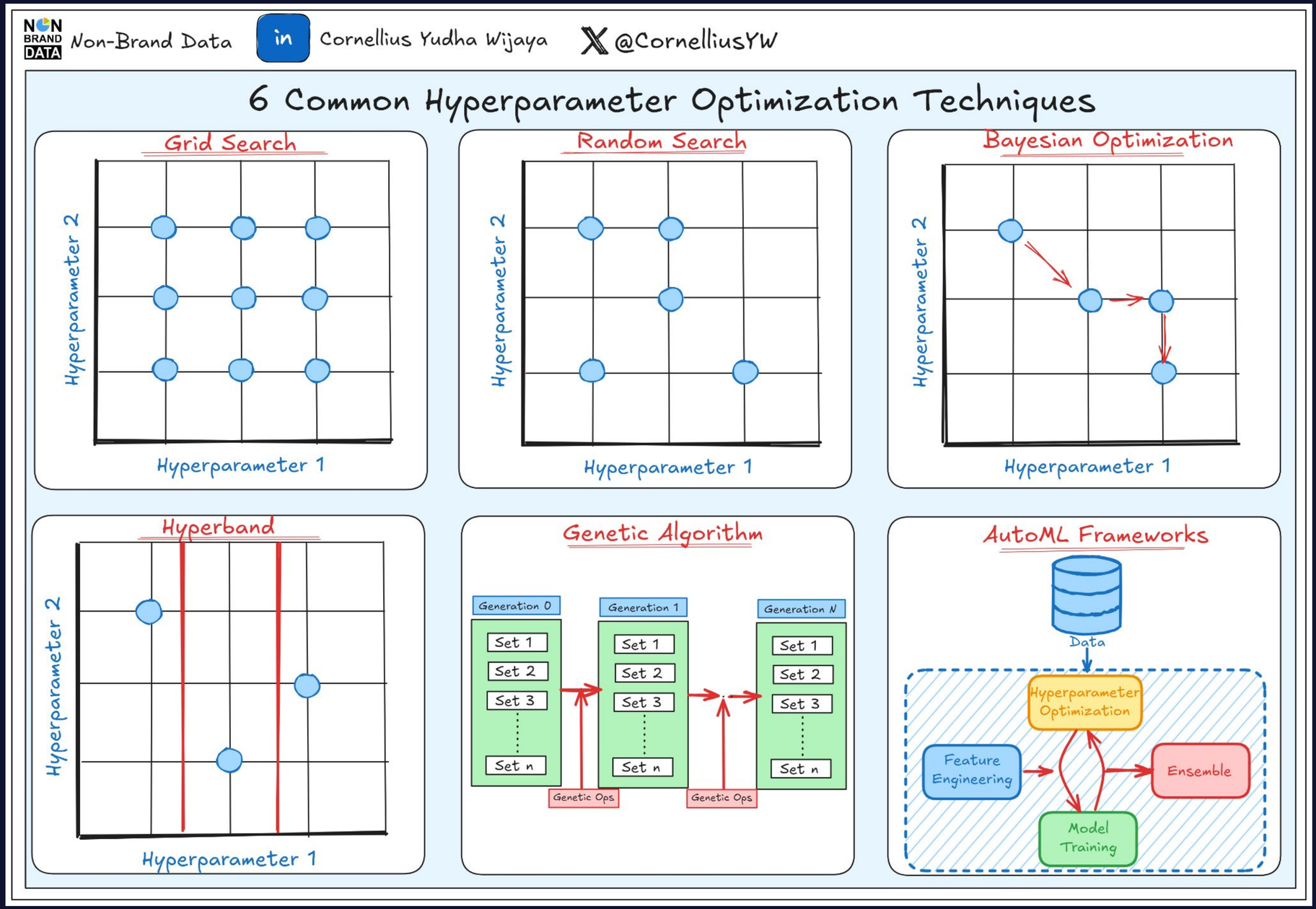




6. Семплинг и балансировка

Метод	Идея	Когда использовать	Плюсы	Минусы
Simple Random	Случайный выбор объектов	Базовый случай, однородные данные	Просто, unbiased (в среднем)	Может сломать баланс классов
Stratified	Выборка по группам (классам) пропорционально	Классификация, дисбаланс классов	Сохраняет распределение	Нужно знать страты
Systematic	Каждый k-й элемент	Большие, случайно перемешанные данные	Быстро, просто	Опасен при периодичности
Cluster	Выбираем группы (кластеры) целиком	Данные по группам (юзеры, города)	Дешево собирать	Высокая дисперсия
Weighted / Importance	Вероятность зависит от веса	Imbalanced datasets, hard examples	Фокус на важных объектах	Может переобучить
Oversampling	Дублируем редкий класс	Сильный дисбаланс	Баланс классов	Overfitting
Undersampling	Убираем частый класс	Большой датасет с дисбалансом	Быстро, уменьшает размер	Теряем данные
SMOTE	Генерим новые объекты (интерполяция)	Малый редкий класс	Лучше чем дубли	Может создавать шум
Bootstrap	Сэмплинг с возвращением	Bagging, Random Forest	Оценка дисперсии	Повторы данных
Time-based	По времени (train < test)	Тайм-серии	Реалистичная валидация	Меньше данных на train

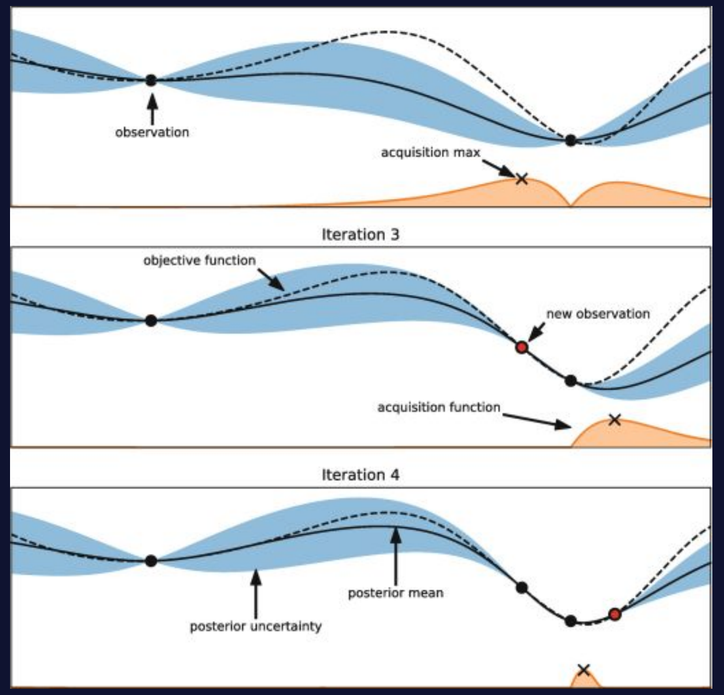
7. Hyperparameter Optimisation



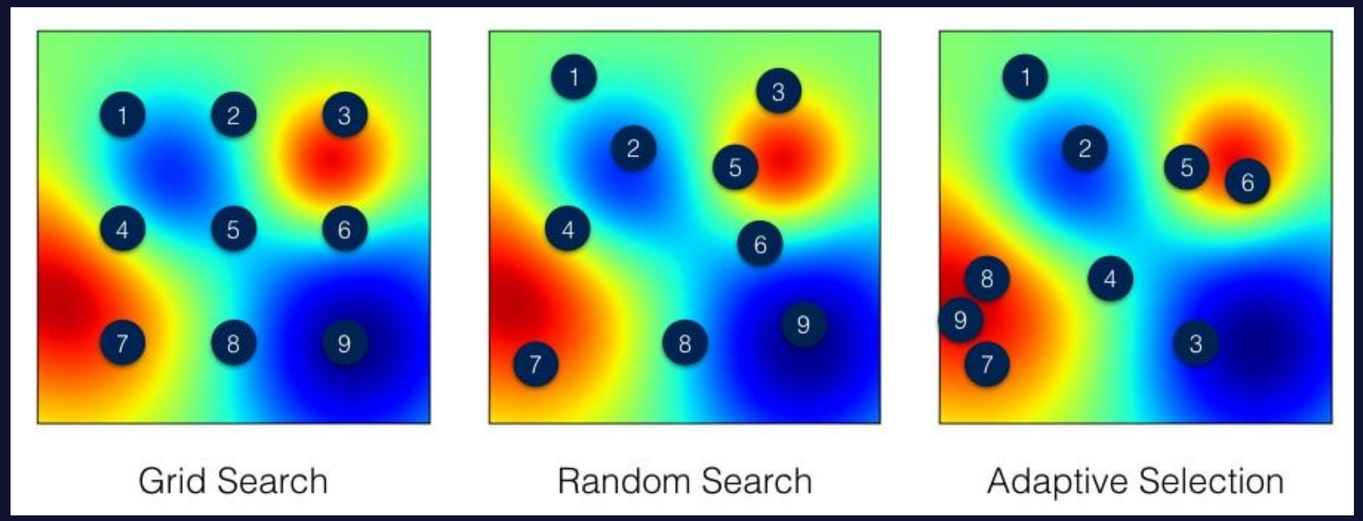
7. Hyperparameter Optimisation

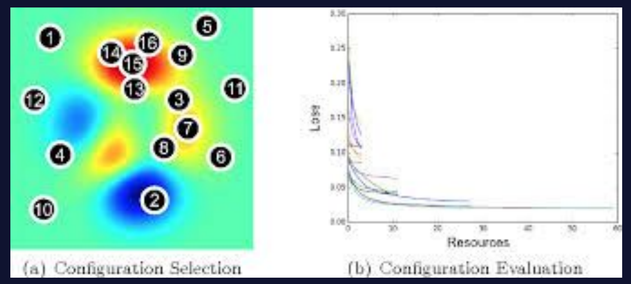
```python
from skopt import BayesSearchCV
from skopt.space import Integer, Real

# Определение пространства поиска гиперпараметров
param_space = {
    'n_estimators': Integer(50, 200),
    'max_depth': Integer(10, 50),
    'min_samples_leaf': Integer(1, 4),
    'max_features': Real(0.1, 1.0, prior='uniform')
}

# Создание модели и настройка с использованием байесовской оптимизации
rf_model = RandomForestClassifier()
bayes_search = BayesSearchCV(rf_model, param_space, n_iter=50, cv=5)
bayes_search.fit(X_train, y_train)

# Вывод наилучших гиперпараметров и оценки
print("Best Hyperparameters:", bayes_search.best_params_)
print("Best Cross-Validation Score:", bayes_search.best_score_)
```

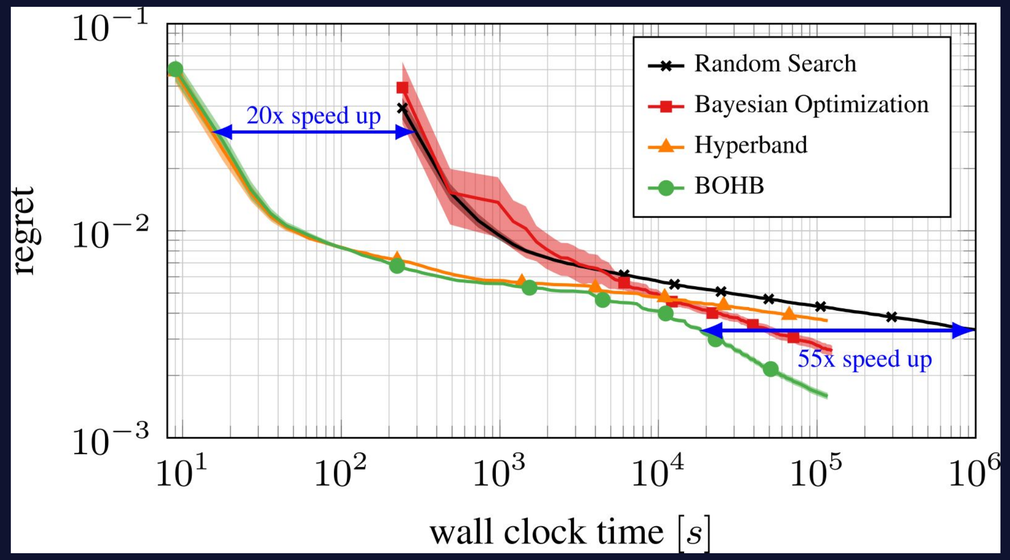
```python
from hyperopt import hp, fmin, tpe, Trials

space = {
    'n_estimators': hp.choice('n_estimators', [50, 100, 150]),
    'max_depth': hp.choice('max_depth', [None, 10, 20]),
    'min_samples_leaf': hp.choice('min_samples_leaf', [1, 2, 4])
}

def objective(params):
    model = RandomForestClassifier(**params)
    score = cross_val_score(model, X_train, y_train, cv=5).mean()
    return -score

trials = Trials()
best = fmin(fn=objective, space=space, algo=tpe.suggest, max_evals=100, trials=trials)
```

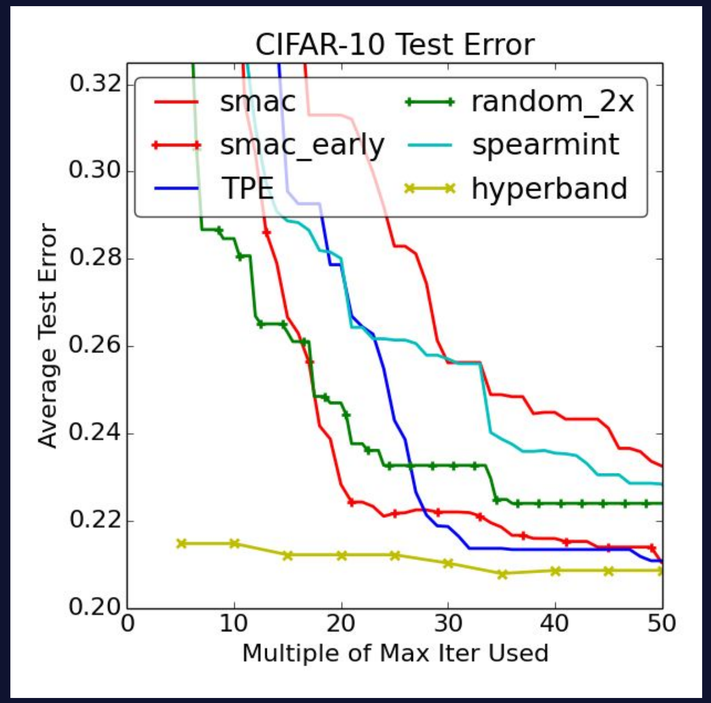


7. Hyperparameter Optimisation









7. Hyperparameter Optimisation

Метод	Суть	Когда использовать	Плюсы	Минусы
Grid Search	Полный перебор всех комбинаций	Малое число гиперпараметров	Просто, гарантированно найдет лучший в сетке	Экспоненциально дорого
Random Search	Случайные точки в пространстве	Когда много гиперпараметров	Эффективнее grid при том же бюджете	Нет гарантии покрытия
Bayesian Optimization	Строит surrogate модель (обычно GP) и выбирает следующую точку умно	Дорогие модели (DL, CV)	Быстро сходится, умный поиск	Сложнее, плохо масштабируется
Hyperband	Ранний отсев плохих конфигов через частичное обучение	Когда можно обучать "частично"	Экономит compute	Нужен понятный budget (epochs)
BOHB	Bayesian + Hyperband	Большие задачи с ограниченным бюджетом	Баланс exploration/exploitation + early stopping	Сложнее в реализации
Evolutionary (GA)	Эволюция популяции гиперпараметров	Очень сложные пространства	Гибкость, не требует гладкости	Медленный, много ресурсов

QA